Core build and migration plan

Status	Draft v1.0, approved plan
Date	2026-10-20
Window	About one month: Oct 21 to Nov 20, then app migration and cutover
Companion documents	`architecture.md`, `04-database-design.md`, `04b-database-notes.md`, `05-api-design.md`

Context

The architecture is settled (`docs/architecture.md`): **Core (Django, `shazlo-server`)** owns identity, discovery and commerce; a separate **vendor-admin service (FastAPI)** is built by teammates in parallel. The user dropped the Oct 30 constraint: about **one month (Oct 21 to Nov 20)**, going all in on the refactor, accelerated by AI, with the **app migrated after** the backend is built. This plan turns the architecture into ordered tasks, a data-migration plan from the old backend, an app-migration plan, and a safe way of working.

Decisions confirmed

- Work happens in the worktree `D:\shazloapp\shaz-core` on branch `refactor/shazlo-server`, with short slice branches merged into it by pull request. `main` (production) is never touched until the end.
- **Live users:** run old and new backends side by side; the new app release uses the new backend; migrate data once at a planned cutover; retire the old backend after old app versions fade.
- **Vendor-admin and portal:** teammates, in parallel. Core builds against the `contracts/` folder and a stub vendor-admin until theirs exists.
- **Prod data:** real users, no real orders (orders are test rows or none). Plan includes a read-only discovery step to confirm counts.
- App API after migration: **GraphQL** (scaffold's choice); the old REST app keeps working on the old backend meanwhile.
- Tools installed: `uv`, Python 3.13.16; scaffold boots; 3 smoke tests, lint and mypy are green; local DB `shazlo_core` exists. **Docker is not running** and the local Postgres 17 has **no pgvector**, so Phase 2 needs the compose Postgres (pgvector image).

Facts that shape the plan (from the audits)

- **Scaffold is ~1,260 lines, mostly config.** Real code: `core` (health, base model, errors, Redis clients, structlog/OTel, request-id middleware), `users` User model (UUID pk, AbstractUser only) and its migration, 3 smoke tests, deploy and config files. **Everything else is a docstring stub.**
- **GraphQL schema has one field** (`serverVersion`), no mutations, no auth context, no persisted-query code. Depth and alias limits exist. Introspection is not disabled in production.
- **Conflicts to remove:** `shazlo/vendors` module and its Ninja routes, the Pinecone integration, pinecone-local in compose, Pinecone env and deps. **Missing:** pgvector dependency, all domain tables, HMAC internal API, outbox/inbox, DB roles, `/internal/*` 404 in Caddy.
- **No CI at all** (no `.github`), no import-linter, no coverage, mypy not strict or in pre-commit, no `check --deploy`.

- **Old backend:** ~40 function-view routes under `/v1`, **no tokens** (client sends `user_id`), passwords hashed with Django `make_password` (so **hashes can be carried over** if the new hasher config matches), `DEBUG=True`, CORS open. Data quirks: text prices, `like_status` mixed (`True/False/"1.5"/"2"`), JSON columns (`embedding`, `preference_vector`, `seen_items`, `images`, `Cart.items`), phone possibly in `email`, `Order` has no quantities, `Brand` stores plaintext bank details, `ItemVariant.__str__` references a removed field. No management commands (items load from CSV upload views).
- **App:** ~20 files and ~28 `fetch` call sites against `${API_BASE_URL}/v1/...`, no shared HTTP client, no token usage, hard-coded `https://api.shazlo.store` in `AuthScreenProfile.jsx` (locally fixed, uncommitted). Auth screens, swipe screens, closets, cart/payment all call the API inline.

Working agreement (how to work cleanly)

1. One slice = one short branch off `refactor/shazlo-server` (for example `core/auth`), one pull request back into it. Merge only when CI is green.
2. Every PR: tests, `schema.graphql` re-exported, docs updated when behaviour changes, migrations included and `makemigrations --check` clean.
3. **Tests first for money, stock and auth;** AI drafts boilerplate, a human reviews every line of those modules. Concurrency and permission tests are mandatory.
4. Build pure logic (taste, slate, state machines, commission, price/stock maths) before wiring it, with golden tests.
5. Parallelise only modules that do not touch (for example feed maths and closets).
6. Keep the contract (`contracts/`, pinned copy in Core) in sync with the vendor-admin team from week 1.

Task plan (Core, in order)

Phase 0: Foundation (Oct 21 to 23)

ID	Task	Done when
0.1	Start Docker; compose Postgres becomes `pgvector/pgvector:pg17` on port 5433; drop pinecone-local	`make up` brings up Postgres, two Redis, MinIO
0.2	Remove `vendors` module, Ninja vendor routes, Pinecone code, deps, env and settings; add `pgvector` dependency; keep Ninja only for webhooks, public and internal routes	`manage.py check` clean; no "vendor" or "pinecone" in core code
0.3	CI workflow `.github/workflows/ci.yml`: uv sync, ruff, mypy, `makemigrations --check`, `check --deploy`, pytest with a pgvector Postgres service, GraphQL schema drift check	CI green on a PR
0.4	import-linter contracts for the module layers (`users`, `auth`, `catalog`, `feed`, `closets`, `orders`, `integrations`, `core`) and mypy in pre-commit	A deliberate bad import fails CI
0.5	Test foundation: `conftest.py`, factory-boy factories, authenticated GraphQL client fixture, fixed depth-limit test	Fixtures used by later phases
0.6	Settings: JWT with audience `core` and key id, HMAC secrets per direction, DB roles (`core_app`, `core_migrate`), disable introspection and GraphiQL in production, async-safe auth middleware	Settings tests pass

Phase 1: Identity and auth (Oct 24 to 28)

ID	Task	Done when
1.1	User model: `phone` (E.164, unique where set), `is_guest`; new models `UserProfile`, `Address`, `Device`, `SocialIdentity`, `Consent`, `RefreshToken`; migrations	Migrations apply on a clean DB
1.2	Auth services: guest start, signup upgrades the same guest, login (email or phone), Google login, refresh rotation with reuse detection, logout, account deletion (anonymise)	Unit and integration tests
1.3	Strawberry context and permission classes from the bearer token	Unauthenticated calls to protected fields fail
1.4	GraphQL: `me`, `publicConfig`, `serviceability`, `guestStart`, `signUp`, `login`, `googleLogin`, `refreshToken`, `logout`, `accountDelete`, `deviceRegister`, address CRUD	Schema exported; CI drift check green
1.5	Rate limits (Redis state) on guest start, login, signup	Limit tests
1.6	Security tests: wrong audience, expired token, refresh reuse, someone else's object returns 404, guest upgrade only with own token; **an old pbkdf2 hash from the legacy DB verifies**	All pass

Phase 2: Catalog and embeddings (Oct 28 to Nov 1)

ID	Task	Done when
2.1	Models `CatalogItem`, `ProductEmbedding` with `CREATE EXTENSION vector` and an HNSW cosine index	Migration applies on the pgvector image
2.2	`import_legacy_items` command: parse text prices to paise, `images` JSON, gender empty string becomes women, `embedding` JSON becomes `vector(512)` (skip bad lengths)	Reconciliation counts match the old DB copy
2.3	Selectors with filters (price, fabric, category, gender, brand); `product` query; public product preview REST	Tests with seeded items
2.4	Minimal inbox and `product.published/updated/unpublished` handler fed by sample events from `contracts/`; embedding job (Modal client, Celery task, retries)	A sample event makes an item feed-eligible

Phase 3: Feed (Nov 1 to 6)

ID	Task	Done when
3.1	Pure `taste.py`, `mix.py`, slate builder, golden tests	Deterministic outputs
3.2	`SwipeEvent`, `UserTasteCluster`, reward events; `swipeRecord` and batch, idempotent on `clientEventId`	Duplicate events ignored
3.3	Feed query: pgvector cosine, filters, seen exclusion from swipe events, 7 plus 3 slate, cold start, explanation	Two users get different slates
3.4	Offline evaluation script and a latency benchmark	Recorded baseline p95

Phase 4: Closets, cart, addresses (Nov 6 to 8)

Models, services, GraphQL, revocable invite tokens, public closet preview, parity tests against old behaviour on seeded data.

Phase 5: Commerce (Nov 8 to 15)

ID	Task	Done when
5.1	`Inventory`, `StockReservation`; pure order state machine; `Order`, `OrderItem`, `OrderStatusEvent` with paise money, commission and price-version snapshot	State machine tests
5.2	`orderPlace`: one transaction (lock rows in id order, reserve stock, create one order per vendor, outbox rows), idempotent	**Concurrency test: two buyers cannot both get the last unit**
5.3	`orderCancel`, return request, order queries; timers (Celery beat): reminder, auto-cancel, expire reservations, unpaid timeout	Timer tests with frozen time
5.4	Payments: cash and UPI on delivery; Razorpay client (create order, signed webhook, refund) behind `online_payments_enabled`; webhook idempotent	Webhook replay test; switch off by default
5.5	Customer push notifications via a Celery task after commit	Task retries on failure

Phase 6: Service contract (draft in week 1, implement Nov 12 to 16)

`OutboxEvent`, `InboxEvent`, `IdempotencyKey`; publisher on Celery beat; HMAC middleware and `/internal/v1` endpoints (confirm, ship, cancel, deliver, payment-collected, refund, set stock, versions, resend, events); Caddy returns 404 for `/internal/*`; contract tests against a stub vendor-admin built from `contracts/` and against the teammates' real service once ready.

Phase 7: Hardening and staging (Nov 16 to 20)

Metrics and traces wired; k6 or locust load test for feed and checkout; `check --deploy` clean; prod compose fixes (migrate job, resource limits, pinned images); staging deploy; backup and restore rehearsal; runbooks.

Data migration plan (old DB to `shazlo_core`)

- **Approach:** idempotent commands `manage.py import_legacy <entity>` that read a **read-only copy** of the old database (separate DB alias or restored dump), never the live DB. Each run reconciles counts and logs rejects.
- **Discovery first (Phase 2 start):** read-only counts and data-quality report on a copy: users (guests vs signed up), phone-in-email rows, bad vector lengths, duplicate emails, action value distribution, empty carts.
- **Mapping** (`docs/04b-database-notes.md` section 4): users to `users` + profile + taste cluster 0 + devices (keep UUIDs; **carry `password` hashes unchanged**; split phone from email by `@`); addresses; scraped items to `catalog_items` + embeddings; `Action` to `swipe_events` (`True` to like, `False` to dislike, `"2"` to cart, `"1.5"` to save); closets with members and items; carts to `cart_items` (default variant); old orders archived (none to migrate); `Brand` exported as JSON/CSV for the vendor-admin team (no passwords, no bank fields).
- **Rehearsals:** two full dry runs on copies; each ends with a reconciliation report (row counts, sampled field checksums, login test for migrated users, swipe and closet counts per user).
- **Cutover (after the app is ready):** announce a short window; freeze writes on the old backend; final delta import; switch the new app release to the new API; keep the old backend serving old app versions; rollback = point traffic back, since the old DB is untouched.
- **Sunset:** retire the old backend when old-version traffic falls below an agreed threshold or a minimum-version check forces updates.

App migration plan (after the backend is stable, from about Nov 14)

1. **A0 foundation:** one GraphQL client (graphql-request on the existing TanStack Query hooks), types generated from `schema.graphql`, token store (refresh and rotate), base URL from build config, remove hard-coded URLs.
2. **A1 auth:** onboarding guest start, signup/login, Google, account delete.
3. **A2 feed and swipes:** feed query, `swipeRecord`, filters, no vectors on the device.
4. **A3 closets and liked items:** closets, invites, share preview, liked list.
5. **A4 cart, addresses, orders, payment:** order screens, stock errors per line, payment methods from `publicConfig`; reconnect Razorpay only when the switch is on.
6. **A5 release:** new app version with a minimum-version check, internal testing then staged rollout, bump `versionCode` above the live 25. Each step is testable on the phone with the existing local setup (Metro, `adb reverse`, new backend on port 8001).

Calendar

Dates	Milestone
Oct 21 to 23	Foundation: CI green, scaffold clean, pgvector up. Contract draft sent to teammates
Oct 24 to Nov 1	Auth done; catalog and importer done; discovery report
Nov 1 to 8	Feed, closets, cart, addresses
Nov 8 to 16	Commerce, outbox/inbox, internal API
Nov 14 to 20	Hardening, staging, rehearsals; app API client and auth screens start
Nov 18 to 20	Buffer. App migration continues about 1 to 2 weeks after, then cutover

Risks and mitigations

Risk	Mitigation
Review and integration become the bottleneck as AI writes fast	Small PRs; mandatory human review for auth, money and stock; contract tests
Vendor-admin team late	Stub built from `contracts/`; Core feeds itself with imported items and sample events
Data surprises in prod	Discovery step and two rehearsals before cutover
Old app versions linger	Run both backends; minimum-version check as the last resort
Pgvector not available locally	Compose Postgres image; documented in the README
Scope creep in the scaffold	Remove vendors, Pinecone and anything not in `architecture.md` first

Verification

- **Per phase:** the "done when" column, enforced by CI (ruff, mypy, import-linter, migration drift, schema drift, tests on pgvector Postgres).
- **Commerce gate:** concurrency test for the last unit, webhook replay, idempotency-key test.
- **Contract gate:** Core and the vendor-admin service both pass the event and command schemas in `contracts/`.

- **Migration gate:** reconciliation report from each rehearsal, including a login test with a migrated legacy password hash.
- **End to end:** place, confirm, ship, deliver and refund across both services on staging, and a phone run of the migrated app against staging.

First actions (next 48 hours)

1. Write this plan into `docs/core-build-plan.md` (with a PDF) on a docs branch in `shaz-core` (done).
2. Start Docker; switch compose to the pgvector image.
3. Phase 0 tasks 0.2, 0.3, 0.4 as three small PRs into `refactor/shazlo-server`.
4. Send the contract draft (`contracts/` internal API and event schemas) to the vendor-admin teammates.
5. Run the read-only data-discovery report on a copy of the production database.